

项目B（Project DPS · qteasy_lab）系统升级改造项目 · 审核资料

版本：2026-08-06（评审修订版） | 所属项目：项目B（系统B · qteasy_lab 投前研究系统） | 项目状态：已通过评审（有条件通过，已落实修订） | 当前实施范围：基础层 + 阶段一
文档性质：提交审核方的项目立项 / 变更审核材料，覆盖升级项目全貌（三阶段），标注当前实施边界。
审核结论：批准执行。采纳项目B第4节目录结构 / 校验机制为最终物理标准；消费回执机制（A 写 `systemA_feedback/consumed_*.json`）获裁定为单向数据流中的确认回执，不属双向写库；B 写端基础层优先实施，A 读端骨架可同步或稍后推进，两端经共享目录契约解耦。

< PAGEBREAK >

一、项目概述

1.1 项目名称与定位

项目	内容
项目名称	项目B 系统升级改造：动态 Beta 计算器 + 前瞻宏观压力仪表盘 + 通用参考维度供应源
所属项目	项目B（系统B · qteasy_lab · Project DPS 投前研究系统）
对接对象	系统A（D:\FF Project, 资产配置与执行系统）
实施边界	基础层 + 阶段一（本审核资料批准后实施）；阶段二 / 阶段三另行立项
总体周期	三阶段、约 12 个月（本资料含三阶段全貌，当前仅就基础层 + 阶段一申请实施）

1.2 项目背景

- 现状：系统B是"静态研究报告生成器"，以 Markdown / HTML / PDF 报告形式向系统A输出研究结论。报告信息密度低、非结构化，系统A无法直接将其作为量化参考维度消费。
- 问题：静态报告与系统A的实际交易决策脱节；系统A的网格 / 资产配置等策略所需的波动率分位、滚动 Beta、宏观压力损益等通用数学参考数据没有稳定供给源。
- 机遇：系统B已具备大量可复用能力——全球宏观评分引擎、滚动波动率 / VaR / ES 指标、交易口径换算、行情多源获取（本地优先 + 在线回退）。缺口仅在于：滚动 Beta 时间序列、通用参考维度结构化输出、前瞻宏观压力仪表盘。

1.3 项目目标

- 将项目B升级为动态 Beta 计算器：输出每资产对多个基准的滚动 Beta 时间序列与当前值。
- 将项目B升级为前瞻宏观压力仪表盘：输出利率 / 期限结构 / 实际利率情景下的对冲效率与条件相关性。
- 将项目B升级为通用参考维度供应源：以结构化文件（CSV / parquet / JSON）向系统A单向输出参考维度，带 `generated_date + data_asof` 元数据头与新鲜度校验。
- 所有输出定位为人工参考：包级 `approval_policy = REFERENCE_ONLY`，风险型字段 `approval_required = true`，永不自动 APPROVED。

1.4 提出日期与状态

- 提出：2026-08-06
- 当前状态：需求澄清完成、技术方案完成、已通过评审（有条件），可进入基础层 + 阶段一实施

< PAGEBREAK >

二、需求与范围

2.1 需求来源（用户诉求要点）

- 系统从"静态研究报告生成器"升级为"动态 Beta 计算器、前瞻宏观压力仪表盘及参考维度供应源"。
- 向系统A输出通用数学参考数据（与具体策略解耦、可复用的波动 / 收益 / 压力维度）。
- 跨系统协同需给出可落地的连接方案与接口契约（系统A侧由人工执行一次性建目录动作）。
- 采用三阶段、约 12 个月的总体推进方式；先建基础层 + 阶段一。

2.2 硬性约束（项目红线）

约束	具体说明
严禁硬编码策略名	机器输出（CSV / parquet / JSON / manifest）不得含"三剑客 / 网格"等系统A中文策略名；只输出通用参考维度。离线分析报告仅使用 <code>strategy_id</code> 标识符
元数据头 + 新鲜度	每包带 <code>generated_date</code> + <code>data_asof</code> ； <code>data_asof</code> 早于当前日期 2 天，系统A可丢弃
永不自动 APPROVED	规则审核必须桌面端人工批准；参考维度 <code>approval_required = true</code> ，不得自动进入交易参数
单向数据流	B → 共享目录 → A；严禁双向写库。唯一例外：B 可只读 A 的日志 / 配置做离线分析

2.3 范围划分

范围	内容
本次审核范围（本资料）	基础层（ <code>reference/包</code> + 既有文件改动）+ 阶段一（B1-1 网格参考表 / B1-2 宏观对冲效率 / B1-3 交易指纹静态分析）
阶段二 / 阶段三	压力模拟器、状态持续期、红黄橙风控建议、参数网格、人机对比月报（本资料仅含方案骨架，见第六章）

2.4 明确不做（Out of Scope）

- 双向写库（共享 SQLite、REST 双向接口均被排除）
- 动态连接（系统A↔系统B 常驻进程或实时通道，沟通机制完善前不落地）
- 自动批准任何规则 / 自动修改系统A参数
- 输出任何绑定系统A策略名的数据

< PAGEBREAK >

三、现状分析

3.1 系统B现有能力（可直接复用）

能力	位置	说明
全球宏观状态识别	core/global_etf_engine.py _macro_state()	输出利率（up/down/stable）、期限结构（inverted/normal）、实际利率（up/down/stable）三元组
投前定量指标	pretrade/metrics.py	滚动波动率（20/60/120/252d）、VaR95 / ES95、最大回撤、静态 Beta / Alpha / 压力相关性
行情多源获取	pretrade/providers.py CompositeProvider	本地 CSV → 研究快照 → SQLite → Tushare → AKShare，自动回退
全球宏观数据	data/processed/global_macro/	DGS10 / DGS2 / DGS30 / DFII10，2003 起，带 available_at 点内时点与质量分级
规则体系（P0 – P5）	评分引擎 + 规则审核	DRAFT / PROVISIONAL / APPROVED / BASELINE / REJECTED 状态机，五档离散 modifier
交易口径换算	core/global_etf_trade_conversion.py	研究评分 → 交易口径独立参考值
桌面端	PySide6 工作台	7 个页面，含宏观规则审核、交易资产映射、数据管理

3.2 系统A对接现状（只读调研）

配置	现状
config/asset_pool.csv	active 资产 14 只（以文件为准，不硬编码数量；与本地行情 13 只重叠，缺 164824.SZ 印度基金LOF）
config/actual_trade_ledger.csv	611 条真实交易流水（另有 CASH 调整 44 条），字段 21 列，全 CONFIRMED，日期 2025-01-02 ~ 2026-08-05
config/manual_override.csv	空表（人工干预记录机制已存在但暂无数据）
config/strategy_params.json	风控阈值（红 / 橙 / 黄回撤、波动率阈值）
strategy_master.csv	6 个策略（strategy_id 标识），含 5 维 pref_ 系列偏好参数（momentum / valuation / volatility / reversal / macro_fit）
调度	两端均无自动化调度，全部手动触发

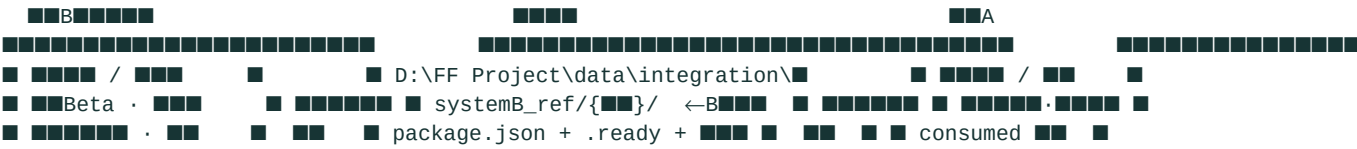
3.3 差距分析

缺口	现状	升级后
滚动 Beta 时间序列	仅 analyze_benchmark() 静态 Beta	20/60/120/252d 多窗口滚动 Beta，多基准
通用参考维度输出	无结构化参考数据导出	CSV / parquet / JSON 数据包，带元数据头
前瞻宏观压力仪表盘	无压力损益维度	四情景对冲效率、条件相关性、压力下行概率
资产池对齐	本地 13 只 vs A 侧 14 只 active	对齐工具 + 缺失标记（quality_level = D，不虚构）

< PAGEBREAK >

四、总体技术方案

4.1 单向数据流架构





• 目录写入权限矩阵（评审确认）：

目录	读写方	说明
systemB_ref/{日期}/	B 读写	B 独占写；A 只读
systemA_feedback/	A 读写	A 独占写消费回执；B 只读用于离线统计（读后只作计数展示，不作分析依据）
backup/	B 读写	B 回滚备份

- 消费回执（Consumed Receipt）语义：A 写 systemA_feedback/consumed_{日期}.json 是单向数据流中的确认回执——B 据此区分"A 没读"与"A 读取出错"，不属于双向写库、不违反单向红线。审核方已裁定采纳。
- B 对 A 的唯一反向接触是只读 A 的 asset_pool.csv / actual_trade_ledger.csv / manual_override.csv / strategy_params.json，且只发生在离线脚本中，输出到系统B自身 reports/，不进入共享目录 systemA_feedback/ 中的 A 反馈文件当作分析依据（避免逻辑循环）。

4.2 跨系统连接方案对比

方案	单向数据流	本地 Windows	日期校验	版本回滚	手动触发	风险 / 成本
纯文件共享目录（推荐）	写/读分工	最佳	元数据头 + manifest	backup/ 快照	零常驻进程	低；零新依赖
共享 SQLite	需双向写库	需 WAL / 锁管理	存表头字段	弱（需另做快照）	需固定 DB 路径	中；易违背单向红线
REST API	可设计	需常驻服务 + 端口 + 认证	需接口层实现	需服务端版本表	每次需启动服务	高；常驻无必要
Git 子模块	可设计	需各自 repo + 凭证	弱	git 历史	每次 commit 摩擦大	高；每日参考数据提交过重

推荐结论：纯文件共享目录。理由：① 两系统同机本地 Windows、均手动触发，无需常驻进程；② 天然单向（写方 / 读方分工），符合红线；③ 每次运行 = 一次目录写入 = 天然版本快照；④ 零新依赖（pandas / json / pyarrow / hashlib 均为现有依赖）。

4.3 共享目录结构

```
D:\FF Project\data\integration\  
  README.md  
  manifest.json  
  systemB_ref\{YYYYMMDD}\  
    package.json  
    .ready  
    grid_reference_table.csv  
    macro_hedge_efficiency.parquet  
    decision_ref_package.json  
    assets_metadata.csv  
  systemA_feedback\  
  backup\{YYYYMMDD}\  
  
← A B B 4.4-4  
← B A  
← B newest_run / runs / checksum / consumed  
← B B  
← schema_version / generated_date / data_asof / files[]  
← + touch  
← B1-1  
← B1-2 pyarrow key-value  
←  
← quality_level = D  
← A A consumed_{}.json / import_log.csv  
← B manifest 30
```

4.4 数据校验与回滚机制

- 1 三重校验：B 先写数据文件 → 逐文件算 sha256 → 写 package.json → touch .ready → 更新 manifest；A 只读带 .ready 的包并重算校验和。
- 2 新鲜度校验：data_asof >= 当前日期 - 2 天才可用（validate_freshness 为 B / A 共用的单一实现，防规则漂移；项目A第 6 节协同硬规则需与本节保持一致）。
- 3 版本回滚：manifest 保留最近 30 个运行版本，A 可指定回滚到任意历史运行目录（从 backup/ 恢复）。
- 4 目录创建兜底（评审确认）：若根目录 D:\FF Project\data\integration\ 不存在，shared_dir.py 主动创建根目录（仅建根目录，不建 A 的子目录 systemA_feedback/），并写入醒目 WARNING 日志"根目录已自动创建，请系统A确认 systemA_feedback 子目录是否存在"。管线不因 A 未建目录而崩溃。

< PAGEBREAK >

五、详细设计

5.1 基础层：新包 qteasy_research/reference/

pyproject.toml 的包发现配置已覆盖新子包，无需新增第三方依赖。

模块	职责	关键实现
config.py	路径 / 常量	SYSTEM_A_ROOT（默认 D:\FF Project, env 可覆盖）、INTEGRATION_DIR、DATA_ASOF_MAX_AGE_DAYS = 2、ROLLING_BETA_WINDOWS、CONE_PERCENTILES、BENCHMARKS 基准映射
metadata.py	元数据头工具	CSV 首行 # generated_date=...; data_asof=...；parquet key-value 元数据；validate_freshness() 单一实现
rolling_beta.py	滚动 Beta	cov / var 逐窗口滚动，var == 0 → NaN；多基准汇总
volatility_cone.py	波动率锥	分位锥 + current_vol_rank + suggest_reference_spread（见 5.2.1）
schema.py	参考数据包	AssetDimensions / DecisionRefPackage（approval_policy = "REFERENCE_ONLY"）
shared_dir.py	共享目录	require_exists / write_run / verify_run / backup_run / list_consumed / rollback_to；根目录缺失时兜底创建根目录（仅根，不建 A 子目录）+ WARNING 日志（见 4.4-4）
asset_pool.py	资产池对齐	读 A asset_pool.csv active 行（不硬编码数量）+ 本地行情优先 + 在线补齐 + 缺失标记
macro_scenarios.py	月度宏观场景	逐月末复用 GlobalEtfEngine._macro_state()；失败月记 macro_unavailable，不自动中性化
pipeline.py	汇总编排	对齐 → 维度 → 组装包 → write_run → backup → verify

5.2 阶段一产出物

5.2.1 B1-1 grid_reference_table.csv（网格参考表）

- 每资产一行（asset 级宽表），复用 metrics.analyze_price_history() 的 volatility_{20,60,120,252}d + 波动率锥分位。
- 关键列：current_vol_{20,60,120,252}d、vol_rank_{20,60,120,252}d、cone_{窗口}_p5/p50/p95、suggested_reference_spread、confidence、data_quality、warnings。
- 参考间距算法：daily_vol = vol / √ 252；基础间距 = daily_vol × 3.0；按 vol_rank 分档乘子（<0.2 → 0.8, <0.4 → 1.0, <0.6 → 1.15, <0.8 → 1.35, 其余 → 1.6），输出价格比例（如 0.012 = 1.2%）。

- 降级策略：历史不足窗口置空 + confidence = low + warning，空值显式写 ""，不让系统A读到 NaN 误判。

5.2.2 B1-2 macro_hedge_efficiency.parquet（宏观对冲效率）

- 四情景（rate_up / rate_down / curve_inverted / real_yield_up）× 每资产，long 格式。
- 逐月末用 macro_scenarios.build_monthly_scenario_table() 打场景标签；macro_ret 用 DGS30 月变化（TLT 代理）与 DFII10 变化（实际利率）。
- 关键列：sample_count、conditional_corr、hedge_efficiency = 1 - min(|corr|, 1)、stress_down_probability、co_movement_probability、avg_monthly_pnl_pct。
- 降级策略：sample_count < 5 置空 + confidence 降级；macro_unavailable 月份不参与。

5.2.3 B1-3 analyze_trader_fingerprint.py（交易指纹 · 静态）

- 性质：静态离线分析，只读 A 的 config/manual_override.csv + config/actual_trade_ledger.csv（A 若新增 data/logs/human_override_log.csv 则一并纳入——路径以项目A第 8 节 A1-3 定义为准，位于 data/logs/ 下而非 config/），输出 Markdown 到系统B reports/trader_fingerprint/。不写共享目录、不写 A 任何文件、不改变两系统运行。
- 内容：逐 strategy_id 计算买卖占比 / 总金额 / 单笔均值 / 现金调整占比 / 持仓天数估计 / 换手率 / 集中度；人工干预档案（override 分布、expires_on 状态）；非 CONFIRMED 记录审计。
- 纪律：报告内只用 strategy_id 标识符，措辞全部面向行为统计，中文策略名零出现。严禁读取共享目录 systemA_feedback/ 中的 A 反馈文件作为分析依据（避免逻辑循环）。

5.3 既有文件改动（最小侵入）

文件	改动
pretrade/metrics.py	新增 rolling_beta() 薄封装（纯逻辑在 reference/rolling_beta.py，避免 metrics 反向依赖 reference 包）；不触碰既有函数
qteasy_research/config.py	新增 SYSTEM_A_ROOT / INTEGRATION_DIR / SYSTEM_A 系列 / TRADER_FINGERPRINT_DIR 常量（复用 PROJECT_ROOT 模式）

不改 global_etf_engine.py（_macro_state 通过实例直接复用）、不改 providers.py、不改 storage.py。

5.4 给系统A侧的指令（接口契约要点）

- 一次性动作：新建 D:\FF Project\data\integration\（空目录即可，B 自动建子目录；若 B 已兜底创建根目录，A 只需确认 systemA_feedback/ 子目录存在）。
- 读侧约定：读 manifest.json → newest_run → 校验 .ready + 日期 + 校验和 → read_csv(..., comment="#") 读 CSV、pd.read_parquet 读 parquet、json.load 读 JSON。
- 消费约定（评审修正）：取消"不写共享目录"限制。A 读包成功后写 systemA_feedback/consumed_{日期}.json 消费回执（integration_reader.py 提供 confirm_consumption(package_date, status="SUCCESS" / "FAILED", reason="") 方法），该回执只写入 systemA_feedback/，绝不触碰 systemB_ref/；属单向数据流确认回执，非双向写库。不得修改 B 写出的任何数据文件。
- 过期规则：data_asof < 今天 - 2 天 丢弃；所有参考维度 approval_required = true，永不自动 APPROVED，须桌面端人工批准后才可进入交易参数。
- 版本回滚：复制 backup/{旧日期}/ 内容并更新 manifest，或让 B 重跑生成。
- 禁止项：A 不得向 systemB_ref/ 写任何文件（双向写库被红线禁止）；B 亦不得把 systemA_feedback/ 当分析依据。

< PAGEBREAK >

六、实施计划

6.1 实施顺序（基础层 + 阶段一）

步骤	内容	依赖
1	基础层 reference / 包 (config → metadata → rolling_beta → volatility_cone → schema → asset_pool → macro_scenarios → shared_dir → pipeline)	无
2	既有文件改动 (metrics.py 薄封装 + config.py 常量)	步骤 1 的 rolling_beta 纯逻辑
3	阶段一脚本 (B1-1 → B1-2 → B1-3 → 决策包 → 端到端管线 → 接口契约生成)	步骤 1 – 2
4	测试 (9 个新测试文件, 离线 fixture, 不依赖真实数据目录)	随各模块并行
5	端到端冒烟 (dry-run 三件套 → 真实写入共享目录 → 系统A确认可读回写 consumed)	步骤 3 – 4
6	文档同步 (SYSTEM_MANUAL / DECISION_LOG 更新)	全程

6.2 阶段二 / 三方案骨架（另行立项）

模块	职责
reference/stress_simulator.py	压力测试损益 (利率 ±50bp / 曲线倒挂 / 实际利率上行 / 滞胀), 历史压力期回放对照
reference/duration_phase.py	宏观状态连续月数统计 (macro_duration_phase), 不绑定策略名
reference/red_flag.py	只读 A strategy_params.json 风控阈值, 产出红 / 橙 / 黄建议, approval_required = true
reference/param_sweep.py	参考间距参数网格扫描 + 换手 / 成本评估
reference/human_machine_compare.py	月度人机对比 (A 实际交易 vs B 参考维度) 月报
版本回滚	shared_dir.rollback_to() 从 backup/ 恢复

阶段二 / 三实施状态追踪（2026-08-07 更新）

模块	立项批次	状态	交付说明
reference/duration_phase.py	阶段二	已完成	9 状态连续月数 + phase (early/mid/late), 并入 macro_regime (提交 0ac463a)
reference/red_flag.py	阶段二	已完成	逐资产红/橙/黄, 镜像 A 回撤/波动率口径, approval_required = true (提交 0ac463a)
版本回滚	阶段二	已完成	rollback_to() 启用 + backup/ 按 run_id 保留最近 30 版 + manifest 同步修剪 (提交 0ac463a)
reference/stress_simulator.py	阶段三	已完成	5 情景历史回放损益 (rate_up_50bp / rate_down_50bp / curve_inverted / real_yield_up / 滞胀组合), 填充 asset.macro_stress ; 样本 <5 降级 None + low 不虚构 (提交 d30cd72)

reference/param_sweep.py	阶段三	已完成	间距网格扫描 + 换手/成本评估；独立 CLI (<code>run_param_sweep.py</code>)，只写 <code>reports/param_sweep/</code> ，不进每日决策包（提交 d30cd72）
reference/human_machine_compare.py	阶段三	延后	月度人机对比月报；依赖 A 侧 <code>human_override_log</code> 积累 ≥3 个月（当前仅 2 行），建议 2026 年 11 月后评估立项

< PAGEBREAK >

七、风险与应对

风险	影响	应对措施
164824.SZ 无本地行情（印度基金LOF）	资产池缺一资产	走在线链路补齐；失败则输出 <code>quality_level = D + warning</code> ，禁止用估算值顶替
新上市 ETF 历史不足（159131 / 588230 / 513650 等）	滚动窗口数据不足	<code>min_periods</code> 降级 + <code>confidence = low</code> ，空值显式写 <code>""</code> / <code>null</code>
纪律风险（单向 / 策略名 / 自动批准）	违反项目红线	代码评审 + 测试断言（机器输出无中文策略名）；接口契约明示禁止项
时钟 / 规则漂移	B / A 侧新鲜度判断不一致	<code>validate_freshness</code> 单一实现 + 接口契约文档同步给 A
共享目录未创建（依赖系统A动作）	无法写入	<code>require_exists()</code> 抛 <code>IntegrationDirMissing</code> 并提示人工动作；降级写 <code>outputs/</code> （dry-run）
系统A配置变动（ <code>asset_pool</code> 增删）	对齐数量偏差	以 <code>asset_pool.csv</code> 文件为准动态读取，不硬编码数量

八、验收标准与验证方式

8.1 单元测试（新增 9 个测试文件，离线 fixture）

测试文件	覆盖点
test_rolling_beta.py	已知协方差验证数值 / 窗口 / <code>var==0</code> →NaN
test_volatility_cone.py	分位数 / 当前分位 / 参考间距边界
test_metadata.py	CSV 头写回读回、parquet 元数据、 <code>validate_freshness</code> 边界（正好 2 天可用 / 3 天丢弃）
test_shared_dir.py	<code>write_run</code> → <code>.ready</code> → <code>manifest</code> → <code>verify</code> → <code>backup</code> → <code>list_consumed</code> 全链路；缺失时 <code>require_exists</code> 抛错
test_asset_pool.py	active 过滤与缺失标记（含 164824.SZ 场景）
test_macro_scenarios.py	月度场景表与 point-in-time 过滤
test_decision_ref_schema.py	包序列化 / <code>approval_policy = "REFERENCE_ONLY"</code>
test_trader_fingerprint.py	买卖比 / 持仓估计 / 无中文策略名断言
test_reference_pipeline.py	端到端：运行目录 <code>.ready</code> / <code>manifest</code> / <code>backup</code> / <code>checksum</code> / 新鲜度

8.2 端到端冒烟（阶段一收尾）

- `run_reference_pipeline.py --dry-run` 检查 `outputs/` 三件套（不写共享目录）。
- 系统A建好共享目录后，指定近 2 日 `--target-date` 跑真实写入，去系统A侧确认可读、可回写 `consumed`。

3 `analyze_trader_fingerprint.py` 产出一份真实流水指纹，人工核对数字。

8.3 可测量验收标准

- 全量测试通过（命令：`python -B -m unittest discover -s "D:\Project DPS\tests" -q`，在 `qteasy_lab` 目录执行）
- 共享目录写入包可在系统A侧按接口契约读取，`data_asof` 新鲜度校验生效
- 系统A成功回写 `consumed` 标记，B 可离线读取
- 机器输出（CSV / parquet / JSON / manifest）经扫描零中文策略名
- 全部参考维度 `approval_required = true`，无任何自动 APPROVED 路径

< PAGEBREAK >

九、决策记录

9.1 用户已确认的决策（2026-08-06）

#	决策	含义
1	共享目录由系统A新建	系统B产出"给A侧指令"（接口契约文档）；并评估跨系统连接方案
2	B1-3 交易指纹先做静态	沟通机制完善前不落地动态连接
3	资产范围对齐系统A active 资产池	以 <code>asset_pool.csv</code> 为准，不硬编码数量
4	先建基础层 + 阶段一	阶段二 / 三另行立项

9.2 关键设计决策

- 曲线倒挂不生成
BASELINE：期限结构倒挂是异常信号，数据窗口内未出现本身就是信息；若未来出现应提醒人工研究，而非用 1.00 掩盖（延续既有"不把缺失当中性"语义）。
- 资产池数量以文件为准：`asset_pool.csv` 当前 active 14 只（与早期描述的 15 只有出入），全部动态读取。
- 缺失不虚构：任何数据缺失（164824.SZ 行情、样本不足、宏观状态不可用）都以 `quality_level = D` / 空值 / warning 显式表达，禁止用估算值顶替。
- 唯一反向接触 = 只读离线分析：B 读 A 配置仅发生在离线指纹脚本，输出到 B 自身 `reports/`，不进入共享目录。
- 评审修订（2026-08-06）：目录创建兜底、权限矩阵、消费回执获裁定、B1-3 读取对象路径核实（`data/logs/human_override_log.csv` 待 A 新增时纳入，见 9.3）。

9.3 评审决议与后续动作（2026-08-06）

#	决议	内容
1	物理标准	以项目B第 4 节目录结构 / 校验机制（ <code>.ready</code> + sha256 + <code>data_asof</code> 2 天新鲜度 + backup 回滚）为最终物理标准；项目A第 6 节协同硬规则与第 4.4 节保持一致
2	反向通道裁定	A 写 <code>systemA_feedback/consumed_*.json</code> 消费回执 = 单向数据流中的确认回执，不属双向写库、不违反单向红线；项目A须取消"不写共享目录"限制并实现回执写入
3	实施解耦	项目B先完成写端基础层（ <code>reference/</code> 包 + <code>shared_dir.py</code> 兜底）；项目A阶段一先搭读端骨架 + 消费回执（即使 B 数据未就绪也可用回退逻辑独立测试），两端经共享目录契约解耦
4	版本对齐	B / A 共用同一 <code>data_asof</code> 新鲜度逻辑（ <code>validate_freshness</code> 单一实现），契约文档同步给 A，防规则漂移
5	目录兜底	B 的 <code>shared_dir.py</code> 在根目录缺失时主动创建根目录（仅根，不建 A 子目录）+ WARNING 日志，管线不崩溃

6	权限矩阵	systemB_ref/ B 读写、systemA_feedback/ A 读写、backup/ B 读写；B 离线指纹分析严禁读取 A 反馈文件，避免逻辑循环
---	------	--

给项目A的修正指令（可直接复制转发）

- 1 修改第 7.1 节：“A 不写共享目录”限制修正为——A 可写共享目录下的 systemA_feedback/ 子目录，用于回传消费状态，不修改 B 的产出物。
- 2 src/integration_reader.py 设计：除 read_decision_package() 外，新增 confirm_consumption(package_date, status="SUCCESS" / "FAILED", reason="") 方法；该方法只写入 data/integration/systemA_feedback/，绝不触碰 systemB_ref/。
- 3 P0-2（集成接口）执行逻辑：读取成功后必须调用 write_consumed_receipt() 写入消费确认文件；P0-3 急停机制增加"读取心跳（heartbeat）判断 B 是否离线"。
- 4 版本对齐：第 6 节协同硬规则的数据_asof 新鲜度阈值与项目B第 4.4 节（2 天）保持一致。

本审核资料已通过评审（有条件，修订已落实）。批准后按第六章实施顺序执行基础层 + 阶段一；阶段二 / 三需另行提交审核。项目A侧按 9.3 修正指令同步执行。